

ECS para un Santuario al estilo Zelda

Se ha decidido plantear un Entity Component System para un santuario de Zelda aprovechando que se está realizando el mismo ejemplo en las prácticas de la asignatura. Un santuario en *The Legend of Zelda* es un espacio sagrado donde el jugador resuelve puzzles utilizando mecánicas físicas, interactuando con objetos y activando mecanismos. Este documento plantea un **Entity Component System (ECS)** para gestionar estas interacciones de forma modular.

1. Mis tres componentes son:

1. **Activador**:

- `tipo` (string): {"presión", "palanca", "energía"}
- `estado` (bool): True/False
- `objetivo_id` (string): ID del receptor vinculado

2. **Receptor**:

- `activaciones_requeridas` (int)
- `contador` (int)
- `accion` (string): "abrir_puerta", "elevar_ascensor", "desbloquear_cofre"...

3. **Temporal**:

- `congelado` (booleano) — si está o no detenido en el tiempo
- `tiempo_congelado_restante` (int) — duración restante en segundos

2. Mis entidades se representan computacionalmente como:

```
entidad = {  
  "id": "unique_id",  
  "grupo": "interruptores" // por ejemplo  
  "componentes": {  
    "Activador": {...},  
    "Receptor": {...},  
    "Temporal": {...}  
  }  
}
```

Donde *entidad* puede ser un nombre como PuertaHabitacion1, InterruptorSalida, PalancaPuerta...

3. Mis componentes se representan computacionalmente como:

```
"Activador": {  
  "tipo": "presión",  
  "estado": false,
```

```

    "objetivo_id": "puerta_1"
  }

  "Receptor": {
    "activaciones_requeridas": 2,
    "contador": 0,
    "accion": "abrir_puerta"
  }

  "Temporal": {
    "congelado": True,
    "tiempo_congelado_restante": 3.0
  }

```

4. Mis sistema se representan computacionalmente como:

```

class_name SistemaActivacion

func update(entidades: Array) -> void:
  for entidad in entidades:
    if "Activador" in entidad["componentes"]:
      var activador = entidad["componentes"]["Activador"]

      if activador["estado"]:
        var objetivo_id = activador["objetivo_id"]
        var receptor_entidad = _buscar_entidad_por_id(entidades,
objetivo_id)

        if receptor_entidad and "Receptor" in
receptor_entidad["componentes"]:
          var receptor = receptor_entidad["componentes"]
["Receptor"]

          receptor["contador"] += 1

          if receptor["contador"] >=
receptor["activaciones_requeridas"]:
            _ejecutar_accion(receptor_entidad)

```

```

class_name SistemaTemporal

func update(entidades: Array, delta: float) -> void:
  for entidad in entidades:
    if "Temporal" in entidad["componentes"]:
      var temporal = entidad["componentes"]["Temporal"]
      if temporal["congelado"]:
        temporal["tiempo_congelado_restante"] -= delta
        if temporal["tiempo_congelado_restante"] <= 0:

```

```
temporal["congelado"] = false
```

5. Las estructuras empleadas son:

- Nodos:
 - SantuarioManager: Singleton que almacena todas las entidades y sistemas.
 - Entidad: Nodos base con ID y sus componentes.
- Scripts:
 - Componentes: Definen propiedades (no tienen lógica)
 - component_temporal.gd
 - component_receptor.gd
 - component_activador.gd
 - Sistemas: Contienen la lógica del juego y modifican los componentes de las entidades.
 - system_temporal.gd
 - system_activacion.gd
 - Santuario: Singleton que une todo. Llama a los sistemas en cada frame
- Métodos clave:
 - entidades.filtrar(tipo_componente): Obtiene las entidades que tienen `tipo_componente` asociado
 - entidades.get(id): Obtiene la entidad con el id
 - update_all(delta): Llama al método `update()` de cada sistema registrado, pasándole todas las entidades y el delta del frame actual.
 - sistema.update(entidades, delta): Método del sistema para ejecutarse constantemente
 - pausar_entidad(id): Setea congelado = True, y un tiempo de congelado restante.
 - entidad_mirada_por_jugador(): Obtiene la entidad que está mirando el jugador lanzando un rayo a la dirección.

6. Pseudo algoritmo que uniría las estructuras

```
// Función que se ejecutaría en bucle
func _process(delta: float) -> void:
    var santuario = SantuarioManager.instance
    santuario.update_all(delta)

    if Input.is_action_just_pressed("activar_parar_tiempo"):
        var entidad = entidad_mirada_por_jugador()
        if entidad and entidad.has("componentes"):
            entidad["componentes"]["Temporal"] = {
                "congelado": true,
                "tiempo_congelado_restante": 5.0
            }
```

donde update_all sería algo como:

```
func update_all(delta: float) -> void:
    for sistema in sistemas:
        sistema.update(entidades, delta)
```

7. Ejemplo de funcionamiento con entidades:

- Entidades:

```
entidades = {
    "puerta_temporal": {
        "id": "puerta_temporal",
        "grupo": "puertas",
        "componentes": {
            "Receptor": {
                "activaciones_requeridas": 1,
                "contador": 0,
                "accion": "abrir_puerta"
            },
            "Temporal": {
                "congelado": False,
                "tiempo_congelado_restante": 0.0
            }
        }
    },
    "interruptor": {
        "id": "interruptor",
        "grupo": "mecanismos",
        "componentes": {
            "Activador": {
                "tipo": "presión",
                "estado": False,
                "objetivo_id": "puerta_temporal"
            }
        }
    }
}
```

- Secuencia:

- El jugador mira a la puerta y usa su poder de congelación -> Se añade el componente Temporal a la puerta.
- El jugador pisa el interruptor.
- Se activa el estado = True del Activador.
- El SistemaActivacion actualiza el Receptor de la puerta.
- La puerta debería abrirse... pero está congelada por el componente Temporal.
- El SistemaTemporal mantiene la puerta sin reaccionar durante 5 segundos.

- Pasados los 5 segundos, se descongela y la acción del receptor (abrir puerta) se ejecuta si todavía es válida.

Durante la secuencia, podemos ver cómo la **asignación dinámica del componente Temporal a la entidad puerta_temporal** modifica el comportamiento del sistema:

1. **Antes de añadir el componente Temporal**, la puerta solo es gestionada por el `SistemaActivacion`, que actualiza su `contador` al recibir una señal de un `Activador`.
2. **Cuando el jugador congela la puerta**, se añade en tiempo real el componente `Temporal`:

```
entidad["componentes"]["Temporal"] = {  
    "congelado": True,  
    "tiempo_congelado_restante": 5.0  
}
```

Esto hace que la entidad empiece a ser procesada por el `SistemaTemporal`, porque ahora tiene el componente correspondiente. Aunque el `SistemaActivacion` siga procesando el `Receptor`, la acción final (abrir la puerta) se bloquea temporalmente, ya que el `SistemaTemporal` impide que se ejecute mientras congelado sea true. Cuando el tiempo de congelación se agota, el `SistemaTemporal` actualiza el estado "congelado" a false.